AFRINEXT · PHASE 3 · BUYER IDENTITY / PROFILE ONBOARDING

The Last Field, and the Three Mutants That Walked Through a Green Suite

`customer_name` now reaches the iPayMoney adapter from data the buyer typed, and a real charge is constructed for the first time. The more useful finding is the second one: the suite proving it was green while three forbidden substitutions passed straight through.

Branch **claude/create-app-repository-1cxsih**	HEAD **22c2ce2**	Unit tests **520 passed**	
Browser tests **31 passed**	Mutations killed **33 / 35**	Real sandbox calls **0**	

READ THIS FIRST — A GREEN SUITE WAS NOT EVIDENCE

The buyer-profile tests passed and **three mutants survived them**: one substituting `display_name` (the synthetic signup address) for a missing customer name, one substituting the phone string, and one inferring the country from the `+227` calling code. Those are three of the rules this milestone exists to enforce.

They survived for a reason worth stating rather than patching around. The profile gate means `initiatePayment` can never reach a charge with a missing name — so a fallback added downstream is **unreachable through the front door**, and every test drives the front door.

The fix was structural, not cosmetic: the metadata construction moved into a function callable with exactly the inputs the gate prevents, and six tests now assert what must *not* happen. **The mutation matrix is what told the difference between a passing test and a proof.**

A Commits and files

Three commits on `claude/create-app-repository-1cxsih`, none merged to `main`. 21 files, +1317 −92.

COMMIT	SUBJECT
f1c3492	Buyer profile: ask the person who they are, once
4f4d7c9	Document the buyer profile, and what it did not change
22c2ce2	Strengthen the provenance tests that mutation testing found hollow

FILE	Δ	WHY
packages/db/migrations/0013_buyer_profile.sql	+41 new	`users.full_name` plus a shape CHECK. Column comments state where the value may and may not come from
packages/db/src/schema/identity.ts	+17	The Drizzle declaration, so CI rebuilds it from zero
packages/core/src/profile/index.ts	+224 new	The domain: load, complete, validate, gate
packages/core/src/profile/profile.test.ts	+301 new	21 tests
packages/core/src/orders/payment.ts	+73 −23	The gate, and `chargeMetadataFor` extracted so the forbidden cases are reachable
.../orders/checkout-provider-wiring.test.ts	+243 −47	Five tests updated to the new truth, six provenance tests added
apps/web/src/components/ProfileGate.tsx	+112 new	The form. Presentation only
apps/web/src/app/[locale]/orders/[id]/page.tsx	+42 −7	Renders the form in place of the pay button when the profile is incomplete
apps/web/src/lib/order-actions.ts	+26 −1	`completeProfileAction`
apps/web/e2e/buyer-profile.ts	+49 new	The shared step every paying journey now passes through
apps/web/e2e/{checkout,delivery,refunds}.spec.ts	+8	Four pay sites, each now completing the profile first
packages/core/src/test/harness.ts	+12 −3	`createTestUser` gains a default profile, and `null` for the state sign-in really leaves
packages/i18n/src/messages/{fr,en}.json	+22 −2	Ten strings each, fr first
docs/architecture/buyer-profile.md	+110 new	The rules, and what was deliberately not built
docs/providers/ipaymoney/*.md	+28 −8	Field sources, assumption A12, and K1–K18 still outstanding

B Where the data lives

On the buyer's own `users` row. **There is no second identity table.**

COLUMN	MEANING
`users.full_name`	**New.** The name the buyer typed. NULL until they do
`users.country_code`	**Not new.** Present since `0001` with a foreign key to `countries`, and never written by anything. It has a writer now

Why a new column rather than `display_name`

`display_name` is **already populated for every existing account** with a value that is not a name — the synthetic address signup mints. It therefore cannot express "we have a real name": reading it would be indistinguishable from reading a placeholder. A column that is NULL until a person types into it can, and `full_name IS NOT NULL` means a human answered.

Overwriting `display_name` would also have destroyed the evidence of what signup did. A test asserts it is left exactly as it was.

There is no `profile_completed_at` flag. Completeness is `full_name IS NOT NULL AND country_code IS NOT NULL` — a derived fact cannot drift from the fields it derives from, and *when* it happened is in the audit log where it belongs.

C The path, end to end

```
buyer signs in (phone OTP)          unchanged — proves a number, nothing else
  └─ user.phoneNumber                verified
     users.full_name    = NULL       nobody has asked yet
     users.country_code = NULL

buyer opens an order, taps pay
  └─ profile incomplete?
       YES → the pay button is REPLACED by the form
              name  ──→ users.full_name
              country ──→ users.country_code   chosen from `countries`
       NO  → pay

initiatePayment()
  ├─ authorize
  ├─ order checks         paid? expired? — the specific truth wins
  ├─ requireCompleteProfile()         ← the gate
  ├─ writes the payment row  (initiated)
  └─ createCharge({
        customer:  { userId, phone },        verified at sign-in
        metadata:  chargeMetadataFor(buyer)  name + country
     })

POST /api/v1/payments  →  { customer_name, msisdn, country, amount, … }
```

Where the gate sits, and why exactly there

Before the payment row is written, and therefore before any provider request could be built. An incomplete profile costs nothing: no row to clean up, no charge that might exist, and nothing for `payments_one_live_per_order` to wedge. The buyer completes the form and pays *the same order*.

After the order checks, so the more specific truth wins. An order that is already paid or expired says so, rather than sending someone to fill in a form that would not have helped.

> **THE FIRST REAL REQUEST**
>
> A test now parses the bytes the adapter addresses to iPayMoney and finds `customer_name`, `msisdn`, the buyer's country rather than the seller's, `XOF`, and `"5000"` — the order's own amount in whole francs. **The transport is injected; nothing reached iPayMoney.** No sandbox credentials exist and no request has ever been made from this repository.

D Security

One buyer cannot touch another's profile, structurally

```
completeBuyerProfile(db, actor, { fullName, country })
                          ↑
                   the subject IS the session
```

There is no user id parameter. Not one that is validated — one that does not exist. So there is no field in any request that names whose row is written, and the class of bug where an id in a form body is trusted *cannot occur here*. A test posts a victim's id through the input object and watches it do nothing; the form carries no identifier at all.

PROPERTY	HOW
A buyer reads and writes only their own profile	The actor is the subject; no id is accepted anywhere
Name and country are never authoritative from the client at payment time	`InitiatePaymentInput` has no such field. A test passes them anyway and they are ignored
The amount stays server-derived	Unchanged — read from the order row inside `initiatePayment`
The synthetic address never leaves Afrinext	Asserted on the request bytes and on the metadata builder
No OTP, code or credential is logged	This path logs none; the audit entry carries the country and a name *length*, never the name
The OTP mechanism is untouched	No change to hashing, rate limiting, Better Auth verification, or the verify request's fields. 26 OTP tests unchanged and green

Privacy of the name itself

The audit log records **that** a profile was completed and **which country** was chosen. It does not record the name: the current value is one column away, and repeating personal data into an append-only table buys nothing. A test asserts the name does not appear in the audit context.

The form tells the buyer, before they type, that the information is sent to the payment provider for the transaction.

E Validating a name without judging it

Surrounding whitespace is trimmed and internal runs collapse to a single space — those are typing accidents. **Nothing else is touched.** No case is forced, no characters are rejected, no assumption is made about how many parts a name has or which script it is written in.

A platform launching in Niger — whose buyers write in French, Hausa, Zarma and Arabic — is the worst possible place to be confidently wrong about what a name may contain. A test walks these through the normaliser and asserts each comes back unchanged:

```
Ali · Mariama Bâ · عائشة محمد · Ngozi Okonjo-Iweala
Jean-Baptiste de La Fontaine · Sœur Marie · O'Brien
Иван Петров · 李小龙 · Þórunn Jónsdóttir · van der BERG
```

The only limits are length — 2 to 120 — which exist to reject an empty string and a pasted document rather than to have an opinion. A `CHECK` constraint enforces the same bounds, so a future code path writing the column directly still cannot store junk; a test proves the constraint fires by attempting exactly that.

The country is validated against the `countries` reference table rather than a regex, so the question actually asked is "is this a country Afrinext knows about" — and an unknown code gets a sentence rather than a foreign-key violation.

F The mutation finding, and why it is the most useful thing here

A new mutation matrix was written for this milestone: nine deliberate defects, each one a rule the brief named. It was run against the first implementation, whose test suite was **fully green**.

#	DEFECT INTRODUCED	FIRST RUN	AFTER STRENGTHENING
P1	The profile gate is removed from `initiatePayment`	KILLED	KILLED
P2	**`display_name` substituted for a missing name**	SURVIVED	KILLED
P3	**The phone string substituted for a missing name**	SURVIVED	KILLED
P3b	The synthetic `@phone.afrinext.local` address sent as the name	—	KILLED
P4	**Country inferred from the `+227` calling code**	SURVIVED	KILLED
P6	A `userId` in the input object is honoured	KILLED	KILLED
P7	Name normalisation removed	KILLED	KILLED
P8	Country no longer checked against the reference table	KILLED	KILLED
P9	An empty name is accepted	KILLED	KILLED

9 KILLED

After strengthening: 9 of 9. Before: 5 of 8, and the three that lived were the three that mattered.

Why they survived a green suite

Because **the gate makes the forbidden branches unreachable through the front door**, and every test drove the front door. `initiatePayment` refuses an incomplete profile before it builds anything, so a fallback added downstream — "if the name is missing, use `display_name`" — never fires in any test that starts from a checkout. The suite proved the happy path thoroughly and said nothing whatever about the forbidden ones.

This is worth dwelling on because the tests were not lazy. They asserted the right things at the wrong altitude: that a buyer *with* a name gets that name, which no substitution mutant contradicts.

What was changed, and what was not

The fix is structural. Metadata construction moved out of `initiatePayment` into `chargeMetadataFor(buyer)`, and the identity loader was exported alongside it. Both can now be called with **exactly the inputs the gate prevents**:

```
chargeMetadataFor({ phone: "+22790123456",
                    fullName: undefined,
                    countryCode: undefined })   → {}
```

Six tests assert what must *not* happen: no name is invented, the phone never substitutes for one, no country is derived from `+227`, and an account in the exact shape real signup leaves — synthetic address in `display_name`, phone string in Better Auth's `name`, `full_name` still empty — yields no name and leaks neither placeholder.

No production behaviour changed in that commit. The gate, the rules and the outputs are identical; what changed is that something now checks them.

> **THE GENERAL LESSON, STATED ONCE**
>
> A guard that makes a bad path unreachable also makes it **untested**. Both facts arrive together, and only the first is comfortable. Where a rule says "never", the test for it has to be able to reach the "never" — which usually means testing one layer below the guard, not through it.

G Results, against the previous baseline

CHECK	RESULT	BASELINE	DELTA EXPLAINED
`pnpm lint`	CLEAN	clean	—
`pnpm typecheck`	CLEAN	clean	—
`pnpm test`	520 PASSED · 16 SKIPPED · 27 files	491 · 16 · 26	**+29:** 21 new profile tests, 6 new provenance tests, 2 new wiring tests. No existing test was removed or weakened
`pnpm build`	COMPILED	compiled	—
`playwright`	31 PASSED	31 passed	Same count. Four pay sites now pass through the profile step first
`reconcile-check`	CLEAN	clean	drift 0 · 0 unbalanced · net XOF 0 · both probes firing
profile mutations	9 / 9 KILLED	—	New matrix. First run: 5 of 8 — see section F
refund mutations	24 / 26 KILLED	24 / 26	Re-run in full. Identical to the baseline: the same 24 killed, the same 2 survivors (R7/R8, redundant by design — R26 kills all three at once), and **every failure count unchanged**
CI	SUCCESS	success	Run #52 on `22c2ce2`: all three jobs green, including the rebuild-from-zero that exercises migration 0013

The 16 skipped are unchanged, and still the same 16

All of them are the real-network iPayMoney tests, gated on `IPAYMONEY_BASE_URL`, `IPAYMONEY_API_KEY` and an explicit `IPAYMONEY_ENVIRONMENT=sandbox`. **The gate was not touched and cannot fall back to Live** — the variable has no default at all. **No real iPayMoney request was made at any point in this milestone.**

Existing suites, all green and unmodified

SUITE	TESTS	CHANGED?
OTP and phone auth	26	UNTOUCHED
Refunds	80	UNTOUCHED
Late payment	31	UNTOUCHED
iPayMoney adapter · webhook	44	UNTOUCHED
Replay	5	UNTOUCHED
Orders and checkout	46	UNTOUCHED
Checkout → provider wiring	21	5 UPDATED, 8 ADDED

The five updated tests asserted the *old* truth — that no name existed and the adapter refused for want of one. One of them was written in the previous milestone with the note: *"when a name becomes available it will fail, and that failure is the signal that the last field landed."* It landed. Its replacement no longer checks that the domain handed the adapter a name; it parses the bytes and finds one.

H What was deliberately not built

NOT DONE	WHY
Settlement	Out of scope and explicitly forbidden. No ledger file, no `ledger/flows.ts`, no `ledger/post.ts`, no settlement hold, no payout logic — none touched
Any OTP or signup change	Forbidden and unnecessary. The name and country are *not* part of the OTP request and never travel with it. Hashing, rate limiting and Better Auth verification are byte-identical; 26 OTP tests unchanged
A payment-channel decision	Explicitly deferred. Nothing defaults `Ipay-Payment-Type`, and the rule that a channel is sent only when explicitly chosen is unchanged
A general profile-editing screen	The form appears where it is needed — at the point of payment — and a buyer corrects what they typed by submitting again. A full account-settings surface is separate work, and this milestone said to find the smallest implementation
KYC	The name is self-declared. iPayMoney documents the field as required and states no verification of it. Recorded as assumption **A12** so nobody later reads it as identity assurance
Any provider-contract change	`ChargeInput` already carried `customer`, `channel` and `metadata`. Nothing was added, and `statesChargeAmount = false` is unchanged
Any refund functionality	No customer-refund API exists in the documentation, and none was invented. `refund()` and `getRefund()` still throw
Merging to `main`	All work is on `claude/create-app-repository-1cxsih`

I Remaining blockers

#	BLOCKER	WHOSE	STATE
S8	No buyer name is collected	Afrinext	CLOSED by this milestone
S7	Checkout does not pass the buyer's details	Afrinext	CLOSED — name, country and phone all flow
S9	**New.** No payment channel is chosen anywhere, and none is defaulted	Product decision	OPEN — DEFERRED BY INSTRUCTION
S6	**No sandbox credentials.** No real payment has ever been made	External	OPEN
S1	Phase 3 has not been approved	You	OPEN
S2	Settlement timing T+N undocumented — K16	iPayMoney	OPEN
S3	The amount is not verifiable from the provider — K8	iPayMoney	OPEN
S4	The 3 % fee's treatment on refund — K11	iPayMoney	OPEN
S5	No customer-refund execution — K1, K5	iPayMoney	OPEN

Sandbox testing is blocked by credentials and by nothing else in the code. The adapter now constructs a complete, well-formed charge request; what it has never done is send one. Two things stand between here and a real sandbox call:

1 Credentials

`IPAYMONEY_BASE_URL`, `IPAYMONEY_API_KEY` and `IPAYMONEY_ENVIRONMENT=sandbox`. Sixteen tests run the moment they exist.

2 A payment channel

The last field. Deferred by instruction, and a charge cannot be built without one because nothing defaults it.

K1–K18 remain outstanding. None was answered, inferred or modified. K17 — whose country `country` means — is still open; Afrinext sends the buyer's self-declared country and labels it `buyerCountry` alongside, so answering K17 changes which fact fills the field rather than requiring anyone to reconstruct what was meant.